Verilog 第五次实验报告

实验名称	32 位乘法器的设计与仿真				
学生姓名	赵宇哲	学号	2411094	指导老师	杨飞
实验地点	公共实验楼 A 区 306 教室		实验时间	2025-10-29 星期三第三四节	

1 实验项目名称

32 位乘法器的设计与仿真

2 实验目的

熟悉 Verilog 中 always 块（always @* 与 always @(posedge clk)）、for 循环的用法。

3 必修或选修

必修

4 实验平台

Xilinx Vivado Design Suite 2017.1（64-bit）

5 实验内容及步骤

5.1 32 位乘法器

实验内容

设计一个模块，参照循环语句中的例子，实现 32 位二进制数的乘法。

设计思路

参照循环语句示例，采用“移位相加”的串行累加结构：在组合逻辑块 `always @*` 中先将被乘数零扩展至 64 位并清零累加器，随后用 `for` 循环自低到高遍历乘数各位；当 `b[i]` 为 1 时，将 `a_ext` 左移 `i` 位后加入累加器，最终得到 64 位乘积。

仿真结果

在 100 MHz 采样时钟下，依次施加定向与随机用例。波形图中：`(a,b)=`
`(00000000,00000000)` 时 `outcome_q=0000000000000000`；
`(00000001,00000001)` 时 `outcome_q=0000000000000001`；
`(FFFFFFFF,00000001)` 时 `outcome_q=00000000FFFFFFFF`；
`(80000000,00000002)` 时 `outcome_q=0000000100000000`。
随后多组随机向量(如 `a=12155324`、`b=C0895E81`)也和参考结果 `golden=a*b` 吻合。采样后的 `outcome_q` 与 `golden` 在每拍均相等，表明乘法器功能正确。

32 位乘法器代码与仿真

5.1.1 设计文件

```v
// mult32.v
`timescale 1ns/1ps
// 32 位二进制数（无符号）乘法 -- 参照循环语句示例：for 循环 + 移位相加（串行累加）
module mult32 (
    input  wire [31:0] a,
    input  wire [31:0] b,
    output reg  [63:0] outcome
);
    integer i;
    reg [63:0] a_ext;

    always @* begin
        a_ext   = {32'b0, a};    // 对被乘数零扩展到 64 位
        outcome = 64'd0;         // 累加器清零
        // 过程性 for 循环：逐位检查乘数 b[i]，为 1 则把 (a<<i) 加入结果
        for (i = 0; i < 32; i = i + 1) begin
            if (b[i]) outcome = outcome + (a_ext << i);
        end
    end
endmodule
```

5.1.2 仿真文件

```v
// tb_mult32.v
`timescale 1ns/1ps
module tb_mult32;
    // 仅用于采样显示的时钟（DUT 仍是组合）
    reg clk; initial clk = 1'b0; always #5 clk = ~clk; // 100 MHz

    // DUT 端口
    reg  [31:0] a, b;
    wire [63:0] outcome;

    // 采样后的结果与期望值（初始化为 0，避免红色 X）
    reg [63:0] outcome_q = 64'd0;
    reg [63:0] golden    = 64'd0;
    integer    k         = 0;

    // 实例化
    mult32 dut(.a(a), .b(b), .outcome(outcome));

    // 采样：避免组合抖动影响观察
    always @(posedge clk) outcome_q <= outcome;

    // 生成波形
    initial begin
        $dumpfile("tb_mult32.vcd");
        $dumpvars(0, tb_mult32);
    end

    // 驱动 + 对拍
    task drive_and_check(input [31:0] aa, input [31:0] bb);
    begin
        @(posedge clk); a <= aa; b <= bb;    // 施加输入
        @(posedge clk);                      // 下一拍采样
        golden = aa * bb;                    // 期望结果（无符号）
        if (outcome_q !== golden) begin
            $display("[%0t] FAIL a=%h b=%h dut=%h exp=%h",
                     $time, aa, bb, outcome_q, golden);
            $fatal(1);
        end
    end
    endtask

    // 主流程
    initial begin
        a = 0; b = 0;
        @(posedge clk);

        // 定向用例
        drive_and_check(32'h0000_0000, 32'h0000_0000);
        drive_and_check(32'h0000_0001, 32'h0000_0001);
        drive_and_check(32'hFFFF_FFFF, 32'h0000_0001);
        drive_and_check(32'h8000_0000, 32'h0000_0002);
        drive_and_check(32'hDEAD_BEEF, 32'hFEED_FACE);

        // 随机回归
        for (k = 0; k < 500; k = k + 1)
            drive_and_check($urandom, $urandom);

        $display("PASS: all tests matched.");
        $finish;
    end
endmodule
```

32 位乘法器代码与仿真

5.1.3 仿真结果

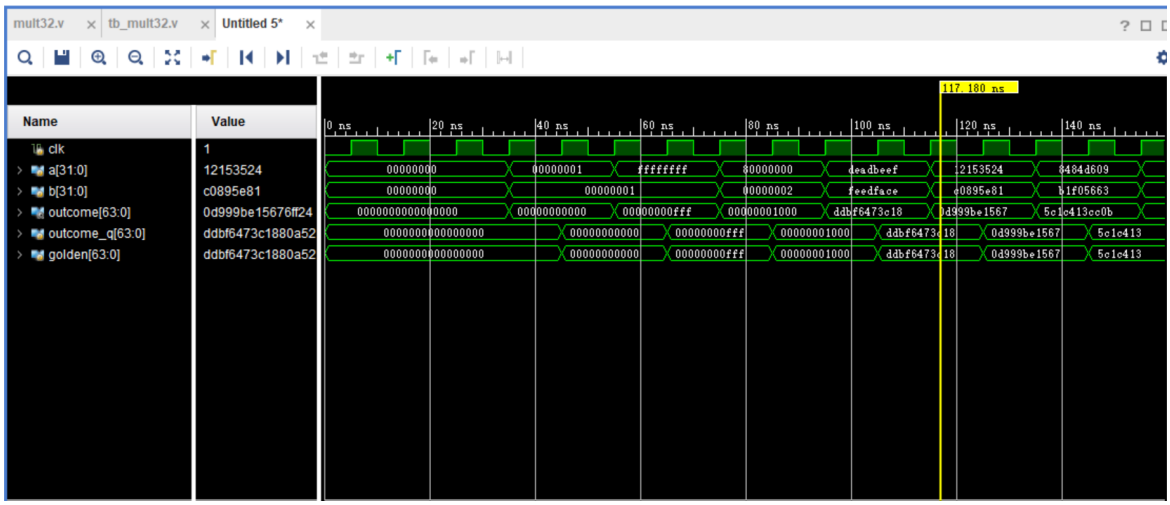


图 1: 32 位乘法器波形图



图 2: 32 位乘法器电路

6 关键问题讨论

本次实现采用在（`always @*` 内用 `for` 将 `(a_ext<<i)` 逐步加到 `outcome`）的方法，每一步都依赖上一步的累加结果。综合后将映射为约 32 级级联加法器，原理图会按从低位到高位顺序展开，整体呈现为一条近似直线。

7 总结

学习了 always 语句写法，有两种不同的触发形式，将之前的几个实验都实现了一遍，编写 testbench 时还是需要注意标点正确使用。作业中实现的 32 位乘法器是在 8 位乘法器的基础上修改的，但是位移过程较为复杂，上网搜索了快速乘法器，有 booth 编码原理和 wallace 树原理，还需要进一步学习。